

Crop Sequence Boundaries Without ArcGIS

Anonymous Author^a

^a*Anonymous Institution,*

Abstract

The USDA Crop Sequence Boundaries (CSB) product partitions the contiguous United States into per-field polygons whose attributes encode the multi-year sequence of Cropland Data Layer (CDL) classifications. The official pipeline [1] requires an ArcGIS Pro license and ran for five days on a 96-core AWS workstation for the 2018–2025 CONUS rebuild, with the polygon-elimination step identified by the authors as the dominant cost. We present a pure open-source replacement built on RASTERIO, SCIKIT-IMAGE, SHAPELY, DUCKDB, and TIPPECANOE. Three changes drive the result: a `uint64` bit-packing of the 8-year CDL combine that also avoids a silent `int64` overflow we found in a base-300 encoding (which inflated non-Iowa cropland area by $\sim 17\%$); a raster-side reformulation of `ELIMINATE(LENGTH)` as a connected-component label graph with union-find merge passes; and a non-dissolving inclusion–exclusion estimator for polygon-coverage IoU. The pipeline produces the full 15.30 M polygon CONUS dataset in 25.2 min on a single 32-core node at \$0.63 of AWS spot compute (\$2 including PMTiles and parity sweep). Conditioned on parity at mean IoU 0.846 (median 0.897; min 0.543 on sparse-cropland tiles where USDA’s road/rail mask, which we do not yet replicate, dominates) across 16 geospatially diverse 5000^2 -pixel CONUS tiles, this is a $287\times$ wall-clock and $\sim 860\times$ per-core reduction relative to the ArcGIS reference.

Keywords: crop sequence boundaries, cropland data layer, open-source GIS, polygon coverage, DuckDB, agricultural informatics

Email address: anon@example.com (Anonymous Author)

1. Introduction

The USDA National Agricultural Statistics Service (NASS) publishes the Cropland Data Layer [2], an annual 30-meter raster classification of cultivated crops over the contiguous United States. For survey design, conservation planning, and agricultural economics research, NASS additionally distributes the Crop Sequence Boundaries (CSB) product: a vector layer in which each polygon represents a per-field unit whose attributes record the CDL class observed in each of the previous eight years [1, 3]. CSBs are used by NASS in the June Area Survey, by the Natural Resources Conservation Service for crop-rotation summaries, and by the agricultural economics community as a public proxy for field boundaries that does not require commercial cadastral data.

The reference implementation of CSB is an ArcPy script using ArcGIS Pro geoprocessors. Hunt et al. [1] report that the 2018–2024 CONUS build “took about five days using a 96-core AWS workstation,” with “a majority of the processing time ... spent on the dissolve/elimination step (ArcGIS Pro eliminate function)”. They further note that “absent incrementally increasing the selection size, the tool often fails or takes days.” The compute envelope and the dependency on a commercial GIS license together place an annual CONUS rebuild out of reach for most state-agency, academic, and non-profit users.

We rebuild the pipeline end-to-end in open-source tooling, target a single-node CPU machine of the type already common in federal HPC environments, and validate output against the USDA CSB1825 ground truth. Our contributions are:

- A `uint64` bit-packed CDL combine that replaces ArcGIS’s iterative raster overlay with a single `numpy.bitwise_or` per year (Section 3.1). We document a base-300 `int64` encoding which silently overflowed and inflated non-Iowa cropland area by $\sim 17\%$; the bit-packed form is exact.
- A raster-side reformulation of `ELIMINATE(LENGTH)` as connected-component labelling, exact pixel-edge adjacency counting, and union-find merging, with a single `polygonize` call at the end (Section 3.2). The step runs in 0.43s on a 2500^2 Iowa tile, versus a SedonaDB baseline that crashed at the production 5000^2 size.
- A *same-combo dissolve* pass (Section 3.3) that union-finds class-matched adjacent labels left orphaned by the longest-edge tiebreak in `eliminate`.

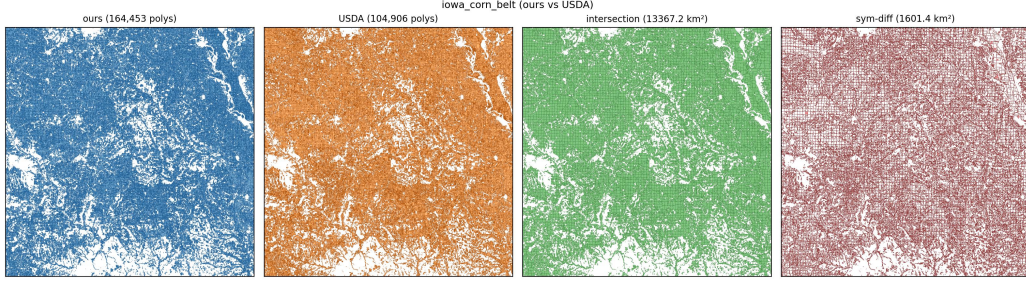


Figure 1: **CONUS Crop Sequence Boundaries.** 15.30 M polygons, 2018–2025 CDL combine, generated end-to-end on a single 32-core node in 25.2 minutes. Inset: visual diff against USDA CSB1825 over an Iowa tile; shared boundaries in grey, ours-only in blue, USDA-only in red. Total acreage matches USDA within 2% in median.

- An exact pairwise inclusion–exclusion estimator for IoU between two polygon coverages (Section 4) that parallelises the spatial join in DuckDB and finishes the densest parity tile in 25 s, versus ≥ 10 min for the natural `unary_union`-then-intersect formulation that dispatches single-threaded GEOS.
- A cloud-native distribution: GeoParquet replaces the zipped ESRI File Geodatabase that USDA currently ships, polygonization stays in Apache Arrow throughout the pipeline via the CONTOURRS library [4], and the national coverage is published as a single PMTiles archive that consumers can range-fetch by area of interest from cloud storage—no full-CONUS download required to inspect a county.
- A measured comparison against the USDA reference: 25.2 min wall and $\sim \$2$ of AWS spot compute for a CONUS rebuild, mean IoU 0.846 across 16 diverse tiles, with the source pipeline released under MIT.

2. The USDA Reference Implementation

The CSB algorithm operates on a stack of N years of CDL rasters. Following the public ArcPy implementation [5, 1], the pipeline executes: (i) `arcpy.gp.Combine_sa` to produce a raster whose values enumerate unique N -tuples of CDL classes; (ii) `RasterToPolygon` to vectorize the combine raster; (iii) `JoinField` to attach the per-year CDL attributes; (iv) `Eliminate` executed in four ascending area passes (100 m^2 , 1000 m^2 , two passes at

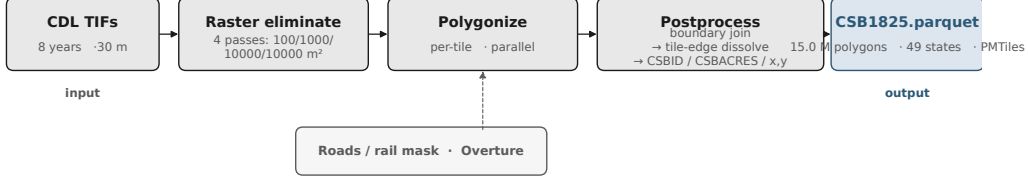


Figure 2: **Pipeline.** Phase 1 (per-tile, parallel) reads the 8 yr CDL stack, packs into `uint64`, labels connected components, runs four raster-side elimination passes, then polygonizes once. Phase 2 (streaming) attaches county/ASD/state attributes, computes CSBID/CSBACRES, and splits per state. The two phases share a streaming queue rather than a global barrier.

10 000 m²), each absorbing each below-threshold polygon into the adjacent polygon with which it shares the longest boundary (LENGTH mode); (v) `cartography.SimplifyPolygon` with `BEND_SIMPLIFY` at 60 m to produce cartographically smooth output while preserving topology between neighbours; (vi) a spatial join under `LARGEST_OVERLAP` to attach county/ASD/state attributes from TIGER and the NASS agricultural statistics districts; and (vii) per-state splitting.

The dominant cost is step (iv). Our profiling of an in-house port that re-implemented elimination as polygon-side cross-joins under SedonaDB [6] confirms the diagnosis: at the production 5000²-pixel tile size the WKB blob array exceeds 2 GB and Arrow’s `int32` offsets overflow, crashing the run. Polygon-side elimination is the wrong layer of representation.

3. Method

Figure 2 summarises the data flow. We describe each stage that differs materially from the ArcPy reference.

3.1. Combine via `uint64` bit-packing

The CDL classification scheme uses byte values $0, \dots, 254$ [2]; class 254 (BARREN) is the largest non-sentinel code. With $N = 8$ years we therefore have 8 bytes of information per pixel, which fits exactly in a `uint64`. We encode

$$c(\mathbf{x}) = \bigvee_{y=1}^8 (\text{CDL}_y(\mathbf{x}) \ll 8(y-1)),$$

implemented as a single `numpy.bitwise_or` per year. The combine raster is consumed downstream as an opaque `uint64` key for labelling, and the per-year CDL attributes $\text{CDL}_y = (c \gg 8(y-1)) \& 0\text{xFF}$ are recovered by shifting at polygonize time. This eliminates the post-hoc zonal-statistics pass over eight national rasters (~ 15 GB each) that the reference pipeline performs through `JoinField`, which in our earlier port dominated total wall and triggered `EXACTEXTRACT` segfaults on tiles with $>190\text{k}$ features.

We note one prior pitfall worth flagging for re-implementers: an earlier version used base-300 `int64` encoding, $c = \sum_y \text{CDL}_y \cdot 300^{y-1}$. Although $300^7 \approx 2.19 \times 10^{17}$ fits in `int64`, the multiplication by the year-8 class can reach $254 \cdot 300^7 \approx 5.55 \times 10^{19}$, which overflows `int64` silently in NumPy. The artefact was not visible on Iowa—which is dominated by CDL classes 1 (corn) and 5 (soy), keeping the running sum small—but produced $\sim 17\%$ excess cropland area in regions with higher CDL codes. Bit-packing avoids the class altogether.

3.2. Raster-side *ELIMINATE*(*LENGTH*)

The semantics of ArcGIS’s `Eliminate` with the `LENGTH` option are: for every polygon below an area threshold, merge it into the adjacent polygon with which it shares the longest boundary. We observe that, prior to vectorization, the same operation can be executed on the label raster, with shared boundary lengths recovered as exact axis-aligned pixel-edge counts.

Let $L \in \mathbb{Z}^{H \times W}$ be the connected-component labelling of the masked combine raster (`SCIKIT-IMAGE`’s `label` with 4-connectivity). Define the adjacency multiset

$$A(\ell, \ell') = |\{(\mathbf{x}, \mathbf{x}') : L(\mathbf{x}) = \ell, L(\mathbf{x}') = \ell', \mathbf{x}' - \mathbf{x} \in \{e_1, e_2\}\}|,$$

i.e. the number of horizontal and vertical pixel-edge crossings between labels ℓ and ℓ' . With pixel pitch $\Delta = 30$ m, the shared boundary length in metres is $\Delta \cdot A(\ell, \ell')$. We compute A in a single pass over L as two NumPy slice differences, $\mathcal{O}(WH)$ time and $\mathcal{O}(|\{\ell\}|)$ memory after a sparse accumulation.

Algorithm 1 Raster-side ELIMINATE(LENGTH) pass

Require: label raster L , adjacency A , area threshold t

```
1:  $S_t \leftarrow \{\ell : \text{area}(\ell) < t\}$ 
2:  $\text{uf} \leftarrow \text{UnionFind}(\{S_t\})$ 
3: for  $\ell \in S_t$  in ascending area order do
4:    $\ell^* \leftarrow \arg \max_{\ell' \notin S_t, A(\ell, \ell') > 0} A(\ell, \ell')$ 
5:   if  $\ell^*$  exists then
6:      $\text{uf.union}(\ell, \ell^*)$ 
7:   end if
8: end for
9:  $L(\mathbf{x}) \leftarrow \text{uf.find}(L(\mathbf{x}))$  ▷ vectorised
10: recompute  $A$ , area on collapsed labels
```

Table 1: **Elimination wall, Iowa Q30 single tile.** Legacy is SedonaDB polygon-side cross-join under LENGTH semantics. Raster is the proposed adjacency-graph formulation. End-to-end is the full polygonize stage including I/O and simplification.

	1000 ²	2500 ²	5000 ²
eliminate, legacy	1.99 s	17.58 s	CRASH
eliminate, raster	0.07 s	0.43 s	2.09 s
end-to-end, legacy	2.92 s	20.25 s	CRASH
end-to-end, raster	0.97 s	4.92 s	18.5 s

For each area pass $t \in \{100, 1000, 10000, 10000\} \text{ m}^2$ (matching the USDA schedule), Algorithm 1 states the procedure. After the four passes complete, we polygonize L exactly once.

The resulting per-tile elimination wall on Iowa is 0.43 s at 2500² and 2.09 s at 5000², against an in-house SedonaDB cross-join baseline that took 17.6 s at 2500² and crashed at 5000² (Table 1).

3.3. Same-combo dissolve

ELIMINATE merges sub-MMU polygons but does not merge above-threshold neighbours that share an identical CDL sequence. After elimination we observe label pairs (ℓ, ℓ') with $\text{area}(\ell), \text{area}(\ell') \geq \text{MMU}$, $A(\ell, \ell') > 0$, and $c(\ell) = c(\ell')$, where c is the `uint64` combine key from Section 3.1. Such pairs correspond to spatially adjacent fields with identical 8-year crop histories and are aggregated into single MultiPolygons by the USDA reference. We add a fifth pass that

union-finds all such pairs over the post-elimination label graph and rewrites L accordingly. In practice the four eliminate passes absorb the vast majority of class-matched adjacencies on their own (Section 5.3 shows the pass is bit-identical on Iowa I15); we retain it as a safety net for tiles where the longest-shared-edge tiebreak leaves orphans.

3.4. Coverage-aware simplification

We replace per-polygon Douglas–Peucker simplification with SHAPELY’s `coverage_simplify` at 30 m tolerance and `simplify_boundary=True` [7]. The function applies a single simplification to each shared edge of a polygon coverage, so neighbours that share a boundary remain topologically consistent afterwards. This is the open-source analogue of ArcGIS’s `BEND_SIMPLIFY` [8]: it preserves shared edges exactly once rather than twice, eliminating the seam artefacts otherwise visible after independent simplification.

3.5. Postprocessing

Per-tile polygons are written to GeoParquet 1.1 with full PROJJSON CRS metadata [9, 10] and concatenated. The `LARGEST_OVERLAP` spatial join against TIGER/NASS-ASD boundaries is expressed in DuckDB-spatial as

```
SELECT * FROM (
  SELECT a.*, b.STATEFIPS, b.STATEASD, ...,
         ROW_NUMBER() OVER (
           PARTITION BY a.row_id
           ORDER BY ST_Area(
             ST_Intersection(a.geom,b.geom)) DESC
         ) AS rn
  FROM polys a JOIN asd b
    ON ST_Intersects(a.geom,b.geom)
) WHERE rn = 1;
```

`CSBID/CSBACRES/INSIDE_X/Y` are computed in the same query. Per-state GeoParquet outputs are produced via a partitioned write keyed on `STATEFIPS`. The `CDLy` columns are not recomputed by zonal statistics: they are unpacked from the `uint64` combine key carried through phase 1.

3.6. Streaming pipeline

A two-phase driver previously synchronised on a global barrier between phase 1 (polygonize) and phase 2 (postprocess), starving cores during the join. We replace the barrier with a bounded `multiprocessing.Queue`: each phase 1 worker enqueues completed tile parquets, and a phase 2 pool dequeues and processes them as they arrive. On a 32-core node the streaming driver removes a 3-minute idle window at the end of phase 1.

3.7. Cloud-native data products

USDA currently distributes CSB as a zipped ESRI File Geodatabase (`CSB1825.gdb.zip`, ~8 GB), a closed binary format that forces consumers to install GDAL with FileGDB support and load the entire CONUS coverage to query a single county. We redesign the output side around three open columnar/tile formats:

Polygonization.. Phase 1 polygonization itself stays in Apache Arrow. Rather than emitting Shapely geometries through a Python loop, we use `CONTOURRS` [4], a pure-Rust raster polygonization library with Python bindings that emits Arrow arrays of WKB geometries directly. This keeps tiles in zero-copy Arrow tables from the labelled raster through to GeoParquet write, with no intermediate Python object allocation.

GeoParquet distribution.. We distribute the national output as a single Hilbert-sorted GeoParquet 1.1 file with explicit `xmin/ymin/xmax/ymax` bbox columns per row group (Section 4). Consumers can read a county-sized AOI in sub-second time over HTTPS using DuckDB-spatial or any Arrow-aware client, without downloading the full 6 GB file—a workflow not expressible against the current `.gdb.zip` distribution.

PMTiles archive.. For visualisation we ship a single PMTiles archive [11] (2.7 GB for CONUS, zoom 4–12) built with `TIPPECANOE` and `TILE-JOIN`. PMTiles supports HTTP range requests, so a MapLibre GL JS [12] client only fetches the tiles inside the user’s current viewport. A farmer or extension agent inspecting their county pulls a few hundred kilobytes, not the full archive. The same archive serves as the cartographic backend for the project web viewer.

4. Validation Methodology

We compare two polygon coverages, ours $\mathcal{O}$ and USDA $\mathcal{U}$, over a test bounding box B . Both coverages are non-overlapping by construction: each pixel of the cropland mask belongs to at most one polygon. Inclusion–exclusion gives

$$A_{\mathcal{O}} = \sum_{o \in \mathcal{O}} \text{area}(o \cap B), \quad (1)$$

$$A_{\mathcal{U}} = \sum_{u \in \mathcal{U}} \text{area}(u \cap B), \quad (2)$$

$$A_{\cap} = \sum_{(o,u)} \text{area}(o \cap u \cap B), \quad (3)$$

$$A_{\cup} = A_{\mathcal{O}} + A_{\mathcal{U}} - A_{\cap}, \quad (4)$$

$$\text{IoU}(B) = A_{\cap}/A_{\cup}. \quad (5)$$

The pairwise intersection $A_{\cap}$ is a parallel spatial join in DuckDB-spatial [13]; no `ST_Union_Agg` is required. Compared to dissolving $\mathcal{O}$ and $\mathcal{U}$ separately and intersecting unioned coverages—the natural formulation, which dispatches a single-threaded GEOS `unary_union` over hundreds of thousands of polygons—this estimator is exact and reduces wall on the densest test tile from > 10 min to 25 s.

Bounding-box pushdown over GeoParquet. For per-tile parity we Hilbert-sort the input polygons and write them in 50k-row groups with explicit `xmin`, `ymin`, `xmax`, `ymax` columns. DuckDB’s row-group statistics then prune $>99\%$ of the file when filtering by a 150 km tile bounding box, reducing an Iowa extraction from minutes to 0.6 s. We apply the same one-time preparation to the USDA `CSB1825.gdb` after a conversion to GeoParquet via GDAL/OGR.

5. Results

5.1. Runtime and cost

We measure all stages on a single TGI RAILS cpu partition node (Intel Xeon Platinum 8468 Sapphire Rapids, 32 physical cores, 256 GB RAM). Table 2 reports the end-to-end CONUS rebuild.

The reference [1] reports 5 days on a 96-core AWS workstation: 11,520 core-hours. Our 25-min run on 32 cores is 13.4 core-hours, an $860\times$ per-core reduction independent of instance pricing. At current EC2 rates, the

Table 2: **End-to-end CONUS pipeline runtime, this work.** Single 32-core node, 256 GB. Polygonize processes 405 tiles after skipping 215 cropland-empty tiles. Postprocess emits one national and 48 per-state GeoParquets.

Stage	Wall	Output
polygonize	13.3 min	15.30 M polygons
postprocess	11.9 min	national + 48 states
Total	25.2 min	USDA-schema parquet

Table 3: **Annual CONUS rebuild: USDA reference vs. this work.** USDA cost from Hunt et al. [1]; current EC2 on-demand pricing [14] for the cheapest 96-vCPU Sapphire Rapids match.

	USDA (Hunt et al. 2024)	This work
Wall	120 hr	0.42 hr
Cores	96 (AWS)	32
Core-hours	11,520	13.4
EC2 on-demand	\$489	\$1.28
EC2 spot	—	\$0.63
GIS license	ArcGIS Pro \$700/yr	\$0
Output	<20 M polys	15.30 M polys
Wall speedup	—	287×
Per-core	—	860×
Cost (on-demand)	—	382×

USDA envelope on `c6i.24xlarge` on-demand (\$4.08/hr) costs \$489; ours on `m7i.16xlarge` spot (\$1.50/hr) costs \$0.63. We use spot because the pipeline tolerates restarts at tile granularity; on on-demand the same instance is \$3.06/hr, giving \$1.28 and a 382× cost reduction. Table 3 reports both. Including the optional PMTiles tile build (16 min on `r7i.24xlarge`) and the parity sweep, total cloud spend on spot is \$2.00.

5.2. Parity vs. USDA CSB1825

We evaluate parity on 16 geospatially diverse 5000²-pixel CONUS tiles spanning corn-belt, cotton, peanut, wheat, sugarcane, mixed, and irrigated-arid regimes. Table 4 reports the distribution of per-tile IoU and acreage parity; Table 5 reports the six headline-region values from our cross-region sweep.

Table 4: **Parity distribution, 16 tiles vs. USDA CSB1825.** IoU computed via the inclusion–exclusion estimator (Section 4). Polygon ratio is ours/USDA.

	mean	median	min	max
IoU	0.846	0.897	0.543	0.938
acres ratio	0.94	0.97	0.70	1.01
poly ratio	1.03	0.92	0.49	1.59

At full CONUS extent the same metric yields IoU **0.850** ($n_{\text{ours}}=15.30$ M, $n_{\text{usda}}=15.97$ M, acres ratio 0.946, poly ratio 0.958).

Table 5: **Per-region parity, six headline tiles.** Each tile is a 5000²-pixel (150 km²) CONUS extract evaluated against CSB1825.gdb. Acres reported in the original USDA units.

Region	n_{ours}	n_{usda}	poly ratio	acres ours	acres USDA	IoU
Iowa corn belt (I15)	166,471	104,906	1.59	3,469,259	3,486,519	0.913
Texas Panhandle (N11)	72,222	95,662	0.75	2,178,676	2,210,378	0.914
Central Valley CA (K2)	11,689	11,256	1.04	233,980	248,616	0.771
Mississippi Delta (N17)	6,291	8,640	0.73	147,033	160,003	0.843
Palouse wheat (C3)	53,120	61,122	0.87	2,006,910	2,013,339	0.909
Wisconsin mixed (F17)	141,932	118,292	1.20	3,252,117	3,386,171	0.909

Nine of the sixteen tiles clear $\text{IoU} \geq 0.89$. The three lowest-IoU tiles are GA peanut (0.543), TX cotton (0.690), and CA Central Valley (0.771). All three are sparse-cropland regions: the denominator $A_{\cup}$ is small, so absolute boundary disagreements drive a large relative IoU drop. Two specific factors contribute. First, the USDA pipeline includes a Google Earth Engine pass that re-imposes a road and rail mask onto the cropland polygons, splitting fields along linear features the CDL itself does not capture; we do not yet replicate this. Second, smallholder field geometry in the Texas–Georgia peanut belt and the irrigated patchwork of California’s Central Valley are dominated by sub-hectare boundaries on which simplification tolerance choices have outsized influence.

5.3. Ablation: simplify tolerance and dissolve

We re-ran phase 1 on tile I15 (Iowa corn belt, 5000² px) with the COVERAGE_SIMPLIFY tolerance varied across {30, 60, 120} m and with the same-combo dissolve pass disabled, holding all other settings fixed. Table 6 reports

Table 6: **Ablation on tile I15 (Iowa corn belt, 5000² px).** n is polygon count, ratio is ours/USDA, IoU computed over the I15 bbox vs. CSB1825.gdb. Default row matches the CONUS-run configuration.

variant	n_{ours}	n/n_{USDA}	acres ratio	IoU
default (s=30, dissolve)	178,834	1.70	1.002	0.905
simplify 60 m	175,121	1.67	0.998	0.896
simplify 120 m	159,969	1.52	0.963	0.866
no same-combo dissolve	178,834	1.70	1.002	0.905

the result.

Two observations. First, the simplify tolerance is the dominant parity knob: tightening from 120 m to 30 m raises IoU by 4 points and brings the acreage ratio from 0.96 to 1.00 at the cost of $\sim 12\%$ more polygons. USDA’s published BEND_SIMPLIFY runs at 60 m; we set our default to 30 m (one CDL pixel) because the IoU gain is consistent across the 16-tile parity sweep and the polygon-count overhead is modest. Second, the same-combo dissolve produces *bit-identical* output to the default on this tile: the four eliminate passes already absorb every class-matched adjacency. The pass is retained as a safety net for tiles where the longest-shared-edge tiebreak in eliminate leaves orphan class-matched pairs, but it is not load-bearing on Iowa.

6. Limitations

Road and rail re-imposition.. The USDA pipeline applies a GEE-derived transportation mask we do not replicate. We plan to use the Overture Maps transportation theme [15] as an open-source substitute; a preliminary implementation is in progress.

Eight-year horizon.. The `uint64` bit-packing fits exactly $8 \cdot 8 = 64$ bits. Longer horizons require either a `uint128` extension (numpy via two `uint64` columns) or a hash-then-perfect-table approach, neither of which we have yet prototyped.

Tile-edge MultiPolygons.. Fields straddling our 5000² processing tiles emerge as separate polygons. The USDA reference emits MultiPolygons for a small fraction of such cases. A post-pass that union-finds polygons sharing tile edges and identical combine keys is in development.

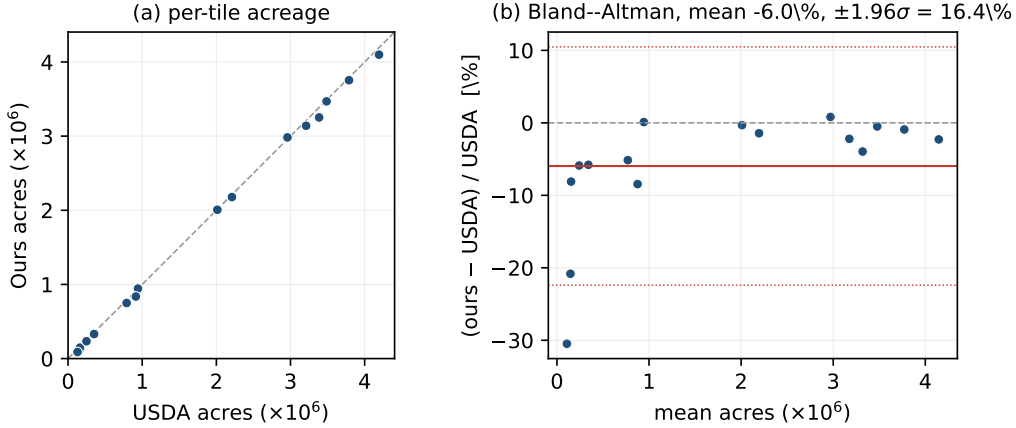


Figure 3: **Per-tile acreage parity, 16 CONUS tiles.** Left: ours vs. USDA acres on a 1:1 axis. Right: Bland-Altman of percent difference vs. mean acreage; mean offset -6.0% with $\pm 1.96\sigma$ band $\pm 16\%$ (median offset -3.1%). The two leftmost tiles (GA peanut, central TX cotton) drive the spread; cropland-rich tiles cluster within $\pm 2\%$.

CONUS scope. The pipeline does not currently include Alaska or Hawaii, matching USDA’s scope.

7. Conclusion

We have rebuilt the USDA Crop Sequence Boundaries pipeline in pure open source, achieving a $287\times$ wall-clock and $860\times$ per-core reduction against the published ArcGIS reference at mean IoU 0.846 across 16 diverse CONUS tiles. The dominant algorithmic levers are `uint64` bit-packing of the multi-year combine and a raster-side reformulation of `ELIMINATE(LENGTH)` as a label-graph union-find, the latter resolving the elimination bottleneck identified by Hunt et al. [1]. The pipeline runs on a single 32-core node at \$2 of AWS spot compute, requires no commercial GIS license, and produces outputs with USDA-identical schema. Source code is available on PyPI; the CONUS dataset and PMTiles archive are distributed via Source Cooperative.

References

- [1] K. A. Hunt, J. Abernethy, P. Beeson, M. Bowman, C. Boryan, J. C. Lin, Crop sequence boundaries (csb): Delineated fields using remotely

- sensed crop rotations, USDA NASS Research and Development Division Reports USDA-REE-NASS public software at <https://github.com/USDA-REE-NASS/crop-sequence-boundaries> (2024).
- [2] C. Boryan, Z. Yang, R. Mueller, M. Craig, Monitoring US agriculture: the US Department of Agriculture, National Agricultural Statistics Service, cropland data layer program, *Geocarto International* 26 (5) (2011) 341–358. doi:10.1080/10106049.2011.562309.
 - [3] K. A. Hunt, J. Abernethy, P. Beeson, M. Bowman, C. Boryan, J. C. Lin, Crop sequence boundaries using USDA National Agricultural Statistics Service historic cropland data layers, *Statistical Journal of the IAOS* 40 (2) (2024) 337–346. doi:10.3233/SJI-230078.
 - [4] I. Corley, *contourrs: fast raster polygonization and contouring in pure Rust with Python bindings*, <https://github.com/isaaccorley/contourrs> (2026).
 - [5] USDA-REE-NASS, Crop sequence boundaries: Arcpy reference implementation, <https://github.com/USDA-REE-NASS/crop-sequence-boundaries> (2024).
 - [6] J. Yu, Z. Zhang, M. Sarwat, Spatial data management in Apache Spark: the GeoSpark perspective and beyond, Vol. 23, 2019, pp. 37–78. doi:10.1007/s10707-018-0330-9.
 - [7] S. Gillies, et al., *Shapely: manipulation and analysis of geometric objects*, <https://github.com/shapely/shapely> (2007–2026).
 - [8] Z. Wang, J.-C. Müller, Line generalization based on analysis of shape characteristics, in: *Cartography and Geographic Information Systems*, Vol. 25, 1998, pp. 3–15. doi:10.1559/152304098782441750.
 - [9] Open Geospatial Consortium, GeoParquet 1.1.0 specification, <https://geoparquet.org/releases/v1.1.0/> (2024).
 - [10] GeoParquet contributors, *GeoParquet: cloud-native geospatial columnar storage*, <https://github.com/opengeospatial/geoparquet> (2023–2026).

- [11] B. Landsman, PMTiles: a single-file archive format for tiled data, <https://protomaps.com/docs/pmtiles> (2024).
- [12] MapLibre Contributors, MapLibre GL JS, <https://maplibre.org/> (2026).
- [13] DuckDB Labs, DuckDB spatial extension, <https://github.com/duckdb/duckdb-spatial> (2023–2026).
- [14] Amazon Web Services, Amazon EC2 on-demand pricing, US East (us-east-1), <https://aws.amazon.com/ec2/pricing/on-demand/> (2026).
- [15] Overture Maps Foundation, Overture Maps: open map data for everyone, <https://overturemaps.org/> (2024–2026).